

Distributed Systems Project

Group: **Comitato Sagra**

Enrico Dalla Croce, Luca Dematté

University of Trento

07 July, 2026

Outline

1. Ask System
2. Update Protocol
3. Ring Topology
4. Election
5. Miscellaneous
6. Conclusions
7. Extras

Ask System

ask()

↓
askSystem.<CSUpdate>ask(

// type of the response

ask()



```
askSystem.<CSUpdate>ask(  
    new CSWriteForward(...),
```

```
// type of the response
```

```
// message to be sent
```

ask()

↓

```
askSystem.<CSUpdate>ask(  
    new CSWriteForward( ... ),           // type of the response  
    replicas.get(this.coordinatorId),    // message to be sent  
                                           // destination
```

ask()



```
askSystem.<CSUpdate>ask(  
    new CSWriteForward( ... ),           // type of the response  
    replicas.get(this.coordinatorId),    // message to be sent  
    timeout,                             // destination  
                                           // timeout defined in ms
```

ask()



```
askSystem.<CSUpdate>ask(  
    new CSWriteForward( ... ),           // type of the response  
    replicas.get(this.coordinatorId),    // message to be sent  
    timeout,                             // destination  
    (res, timedOut) → {                  // timeout defined in ms  
                                          // response handling:
```


ask()



```
askSystem.<CSUpdate>ask(  
    new CSWriteForward( ... ),  
    replicas.get(this.coordinatorId),  
    timeout,  
    (res, timedOut) → {  
        if (!timedOut) {...}  
    })  
    // type of the response  
    // message to be sent  
    // destination  
    // timeout defined in ms  
    // response handling:  
    // - when the response arrives
```

ask()



```
askSystem.<CSUpdate>ask(  
    new CSWriteForward( ... ),  
    replicas.get(this.coordinatorId),  
    timeout,  
    (res, timedOut) → {  
        if (!timedOut) { ... }  
        else {...}  
    })  
    // type of the response  
    // message to be sent  
    // destination  
    // timeout defined in ms  
    // response handling:  
    // - when the response arrives  
    // - in case of timeout
```

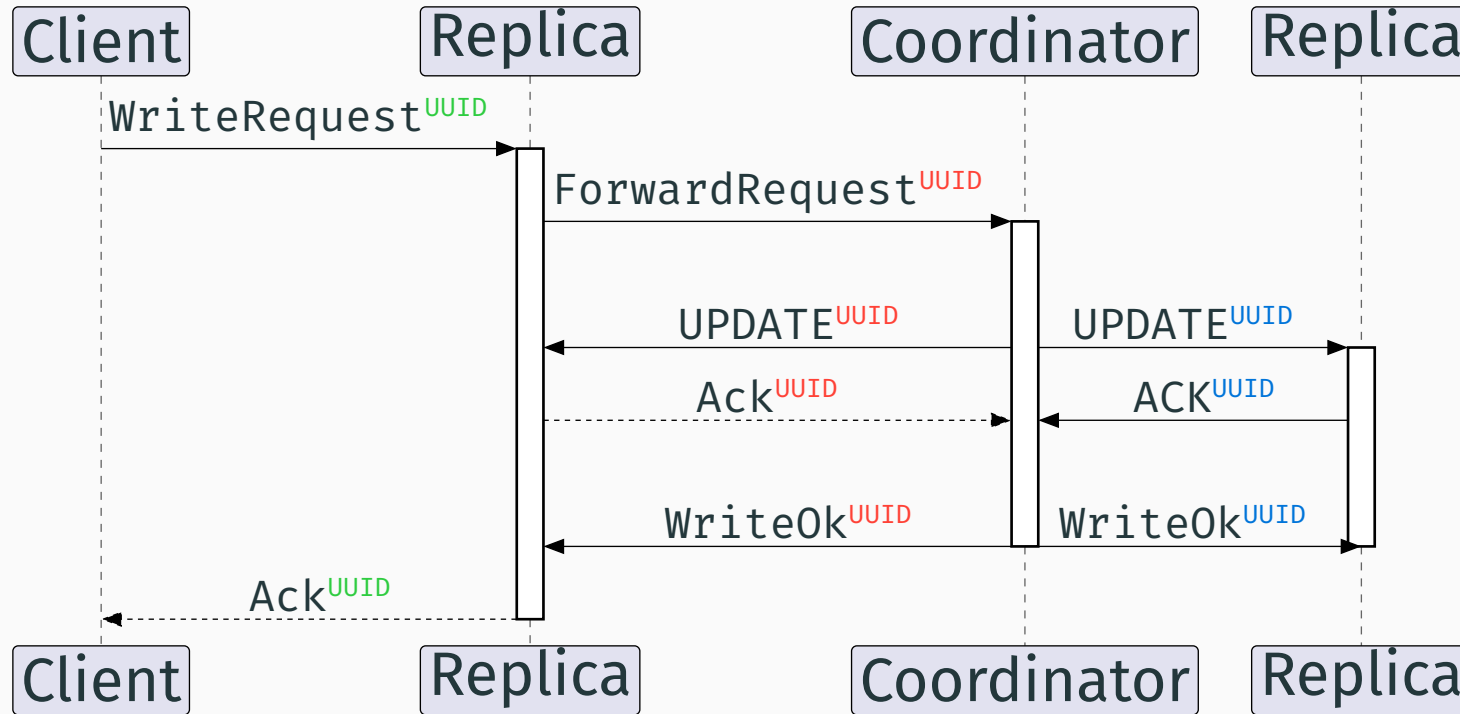
ask()

↓

```
askSystem.<CSUpdate>ask(  
    new CSWriteForward(...),  
    replicas.get(this.coordinatorId),  
    timeout,  
    (res, timedOut) → {  
        if (!timedOut) {...}  
        else {...}  
    })  
}
```

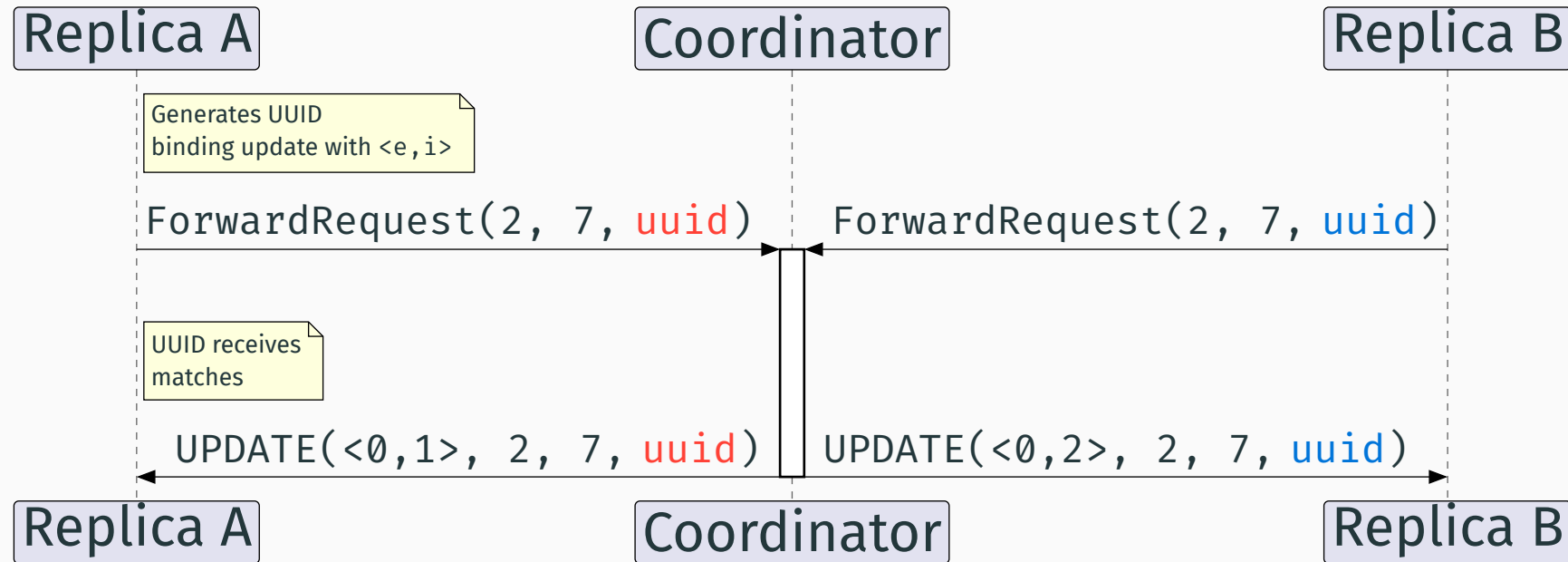
// type of the response
// message to be sent
// destination
// timeout defined in ms
// response handling:
// - when the response arrives
// - in case of timeout

Example

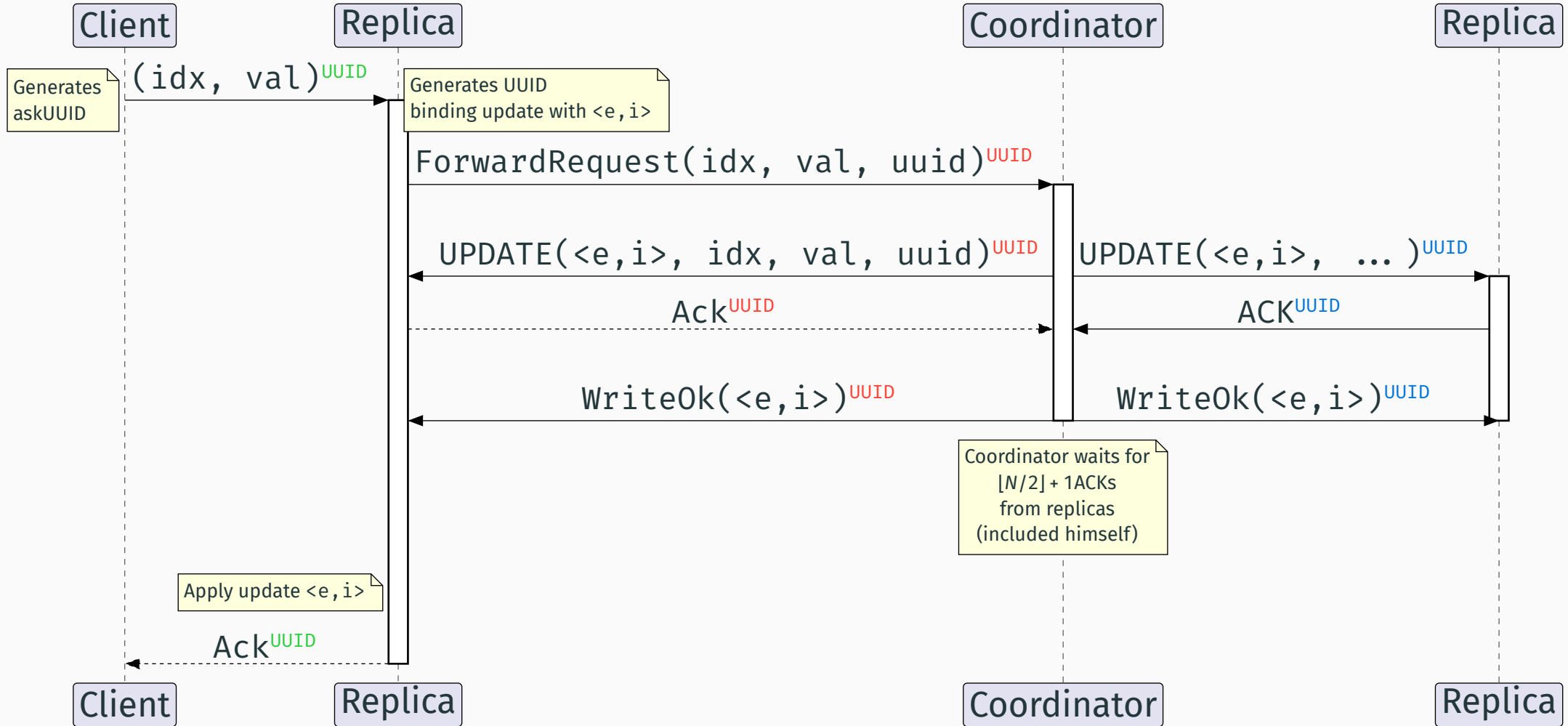


Update Protocol

UUID to distinguish WriteRequest(s)

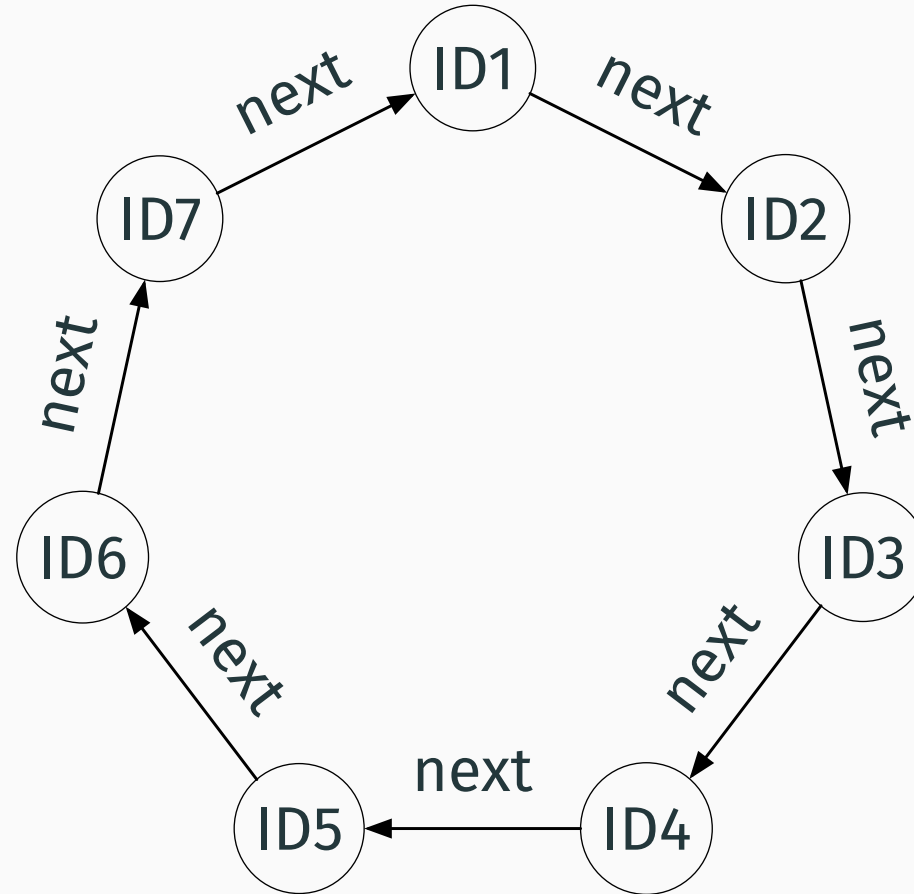


Overview



Ring Topology

Next



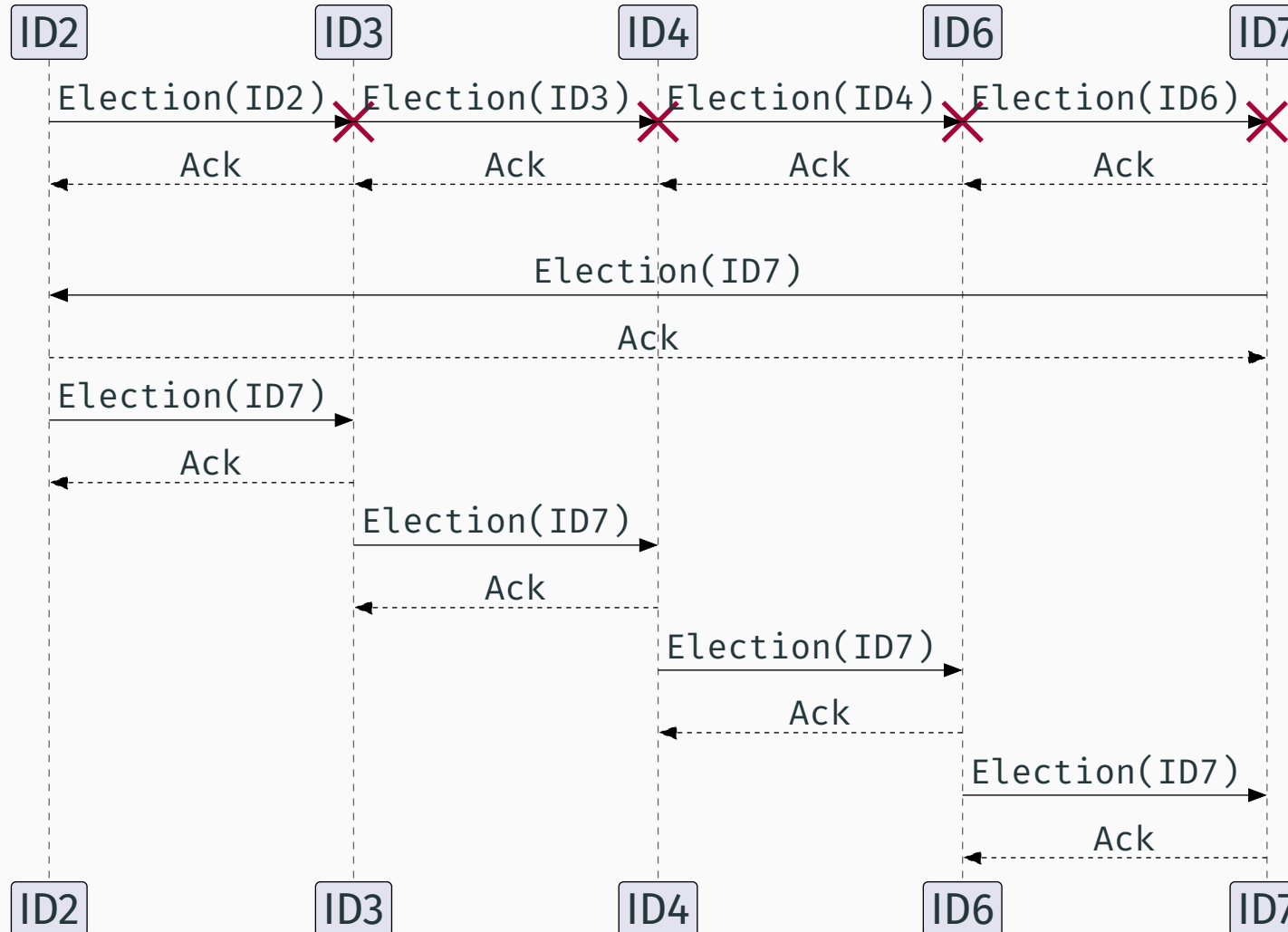
Election



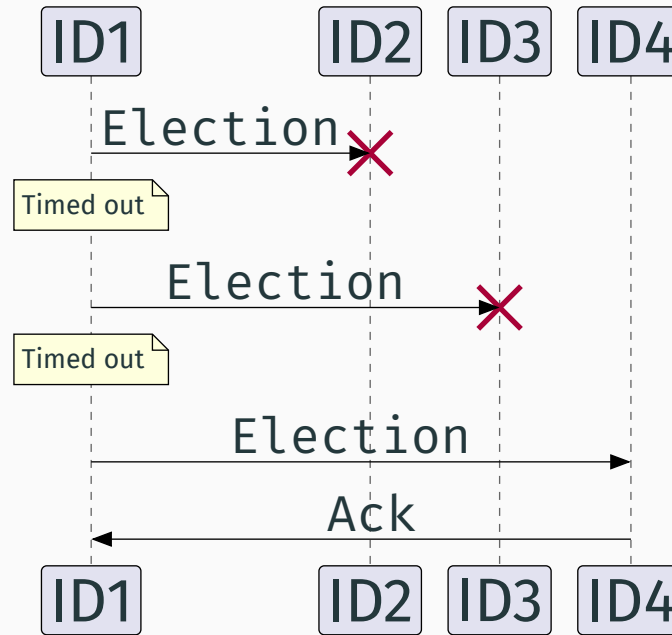
Non Blocking Election

To make sure that the election process terminates we do 4 things

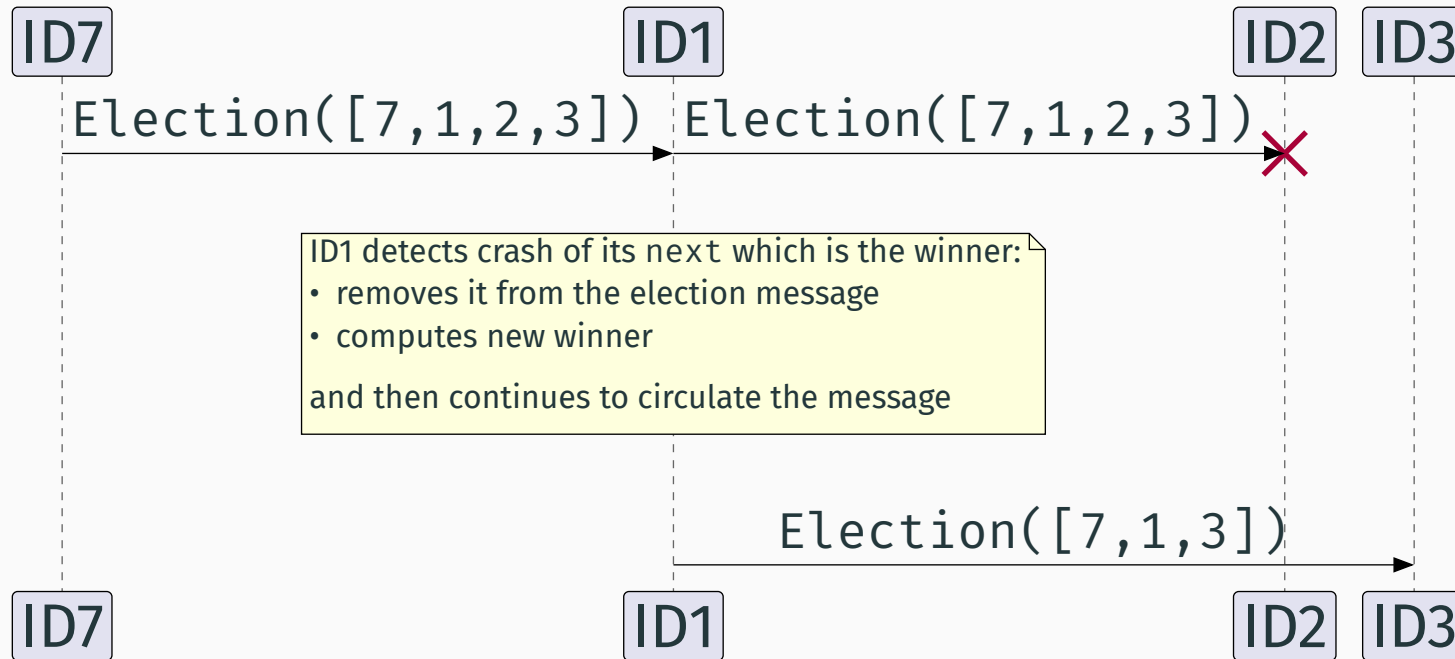
I - Duplicated Election messages



II - Next Crash



III - Winner Crash



IV - Catch all solution

Every replica start a timer at the start of every election.

If the timer expires the replica starts a new election attempt.

In a similar fashion to when handling multiple replicas trying to start the election, only the most recent attempt is considered:

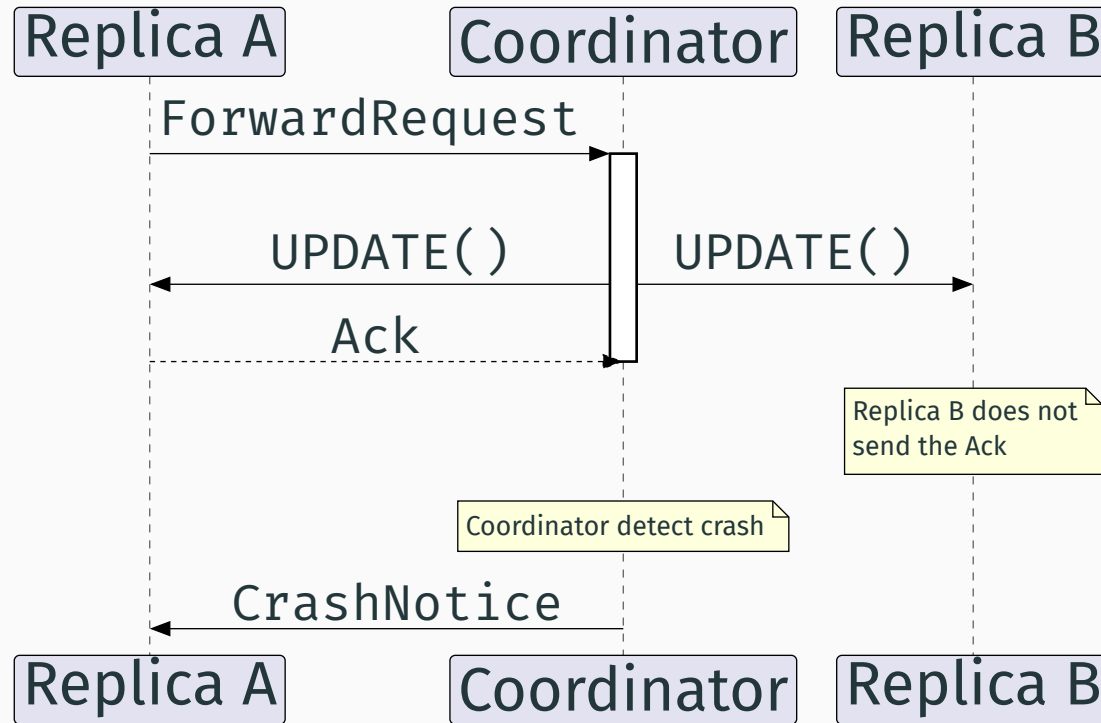
```
if(  
    msg.getElectionAttempt() ≤ this.electionAttempt) {  
    //drop election message  
} else {  
    //accept election message  
}
```

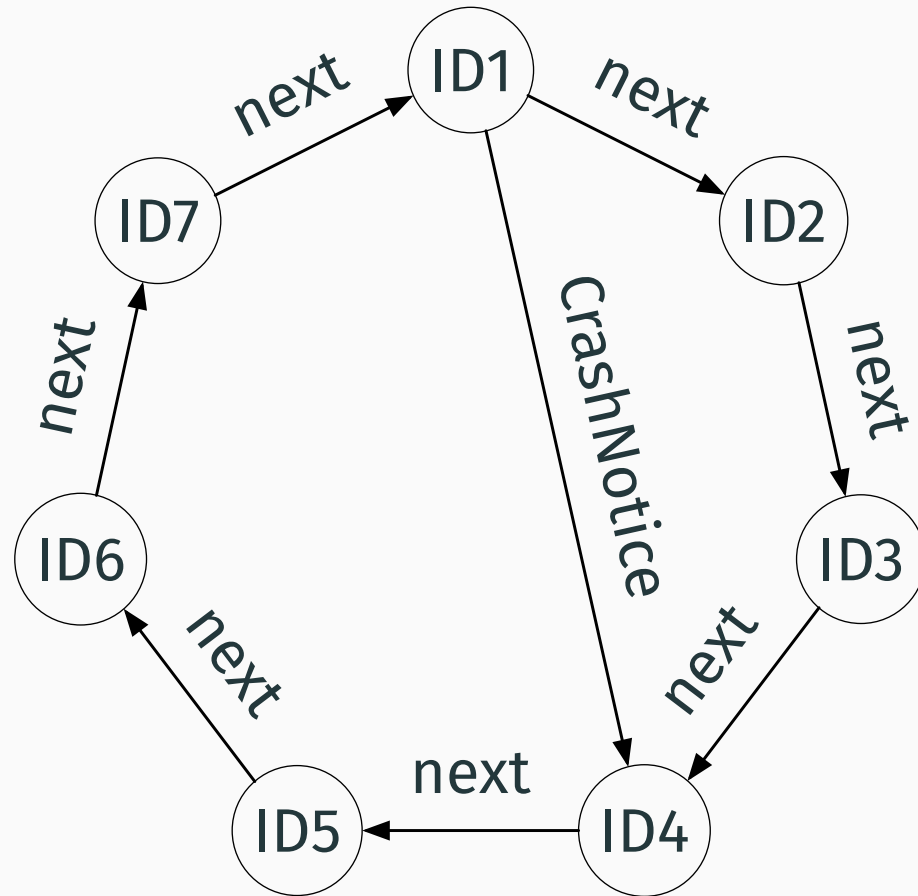
Miscellaneous

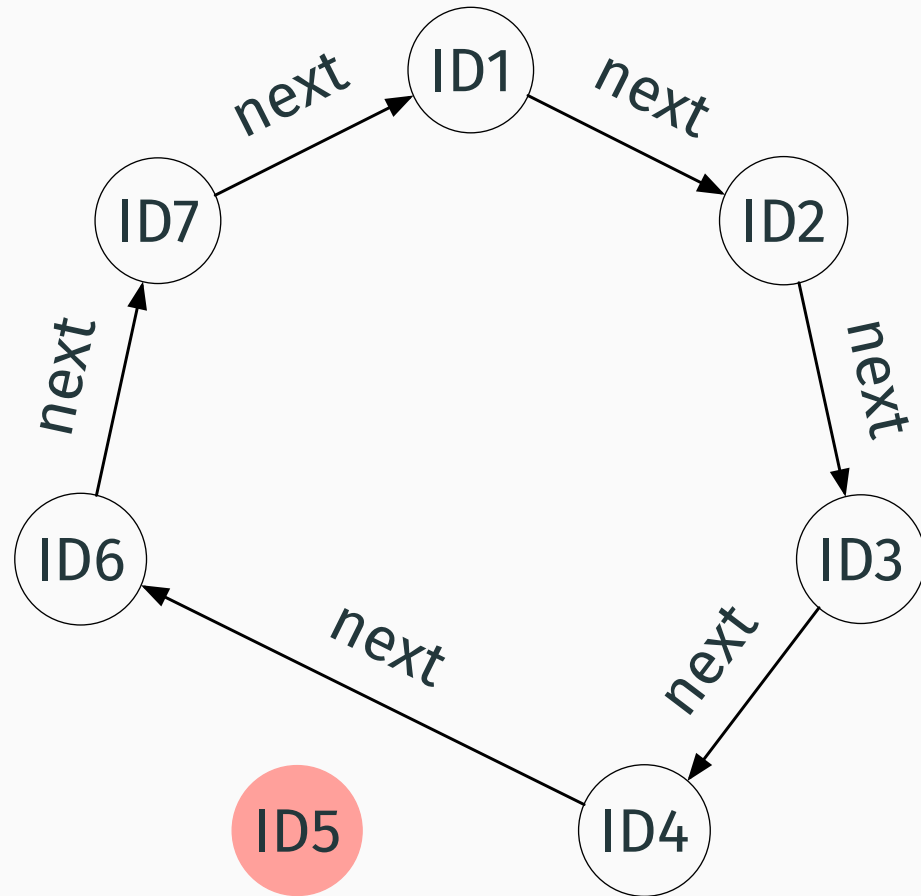
Non Blocking updates

During the election `WriteRequest` are still received but processed when the election is over

Crash Notice







Conclusions

Shower Thoughts

- It was very interesting seeing how such a simple protocol can present so many problems in a distributed implementation.
- We spent a lot of time ironing out all edge cases we encountered, so we are confident about our implementation, but there might be more bugs that we missed.

About AskSystem

It was very fun to build it. But definitely not necessary.
For sure we improved our knowledge of Java.

Thank you for your attention

Questions?

Extras

getNextOf()

```
private int getNextOf(int replicaId) {    replicaId=4
```

getNextOf()

```
private int getNextOf(int replicaId) {      replicaId=4  
    return this.replicas.keySet().stream() [1] [2] [3] [4] [5] [6] [7]
```

getNextOf()

```
private int getNextOf(int replicaId) {      replicaId=4
    return this.replicas.keySet().stream() [1] [2] [3] [4] [5] [6] [7]
        .filter(id → id > replicaId)      [5] [6] [7]
```

getNextOf()

```
private int getNextOf(int replicaId) {      replicaId=4
    return this.replicas.keySet().stream() [1] [2] [3] [4] [5] [6] [7]
        .filter(id → id > replicaId)      [5] [6] [7]
        .min(Integer::compare)           [5]
```

getNextOf()

```
private int getNextOf(int replicaId) {      replicaId=4
    return this.replicas.keySet().stream() [1] [2] [3] [4] [5] [6] [7]
        .filter(id → id > replicaId)      [5] [6] [7]
        .min(Integer::compare)            [5]
        .orElse(Collections.min(this.replicas.keySet()));
}
```